

COMPUTER VISION

Five Tasks in Understanding an Image

From a single label to a pixel-perfect map of every object

1 Classification



2 Localization



3 Detection



4 Semantic Seg.



5 Instance Seg.

What Can We Ask a Computer to Understand from an Image?

A computer sees an image only as numbers. Our goal is to turn that numerical data into meaningful information — but the type of information depends on the question we ask.



ONE IMAGE

Many possible questions

- 1 What is in the image?**
- 2 Where is the object?**
- 3 How many objects are there?**
- 4 Which pixels belong to each class?**
- 5 Which pixels belong to each individual object?**

Five Computer Vision Tasks

Each task asks a more detailed question about the same image.



Increasing level of detail →

Five Computer Vision Tasks

Each task asks a more detailed question about the same image.

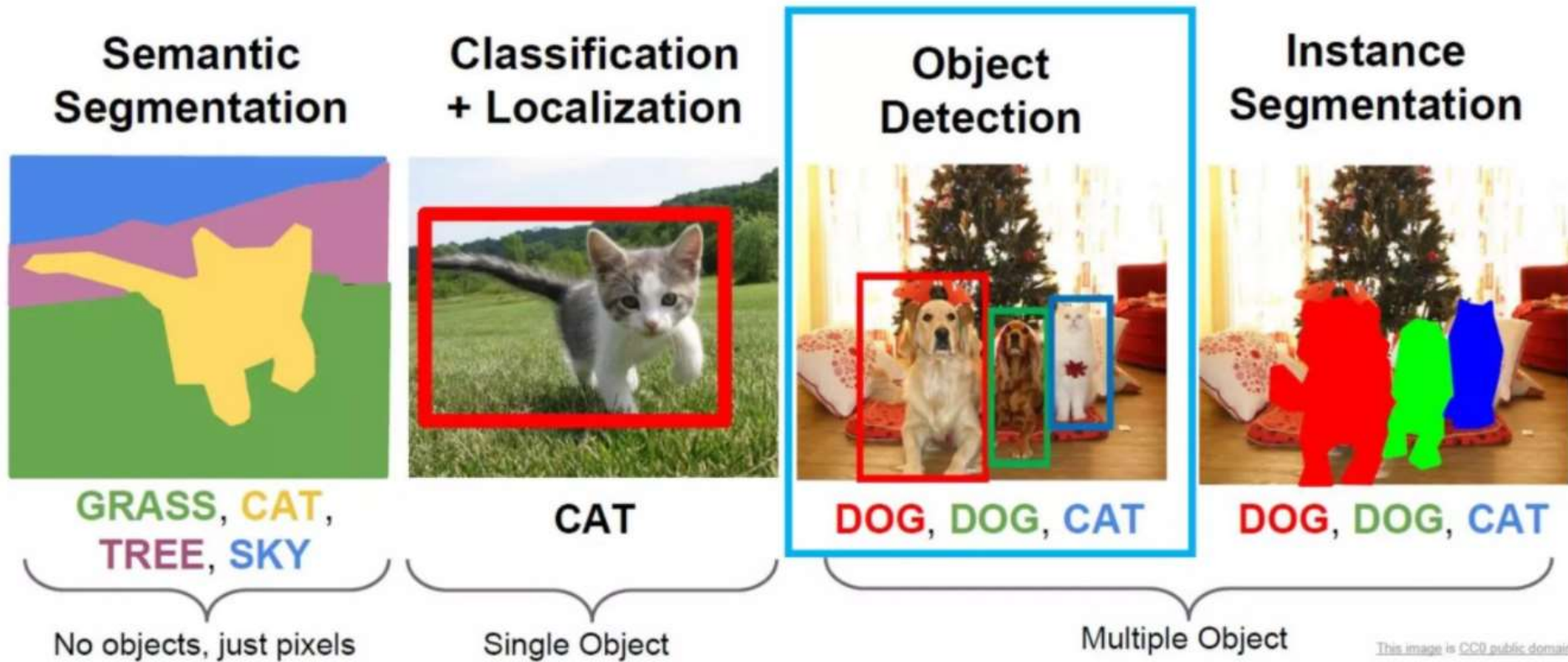




Image Classification

Main question: What is in the image?

Image classification assigns one label — or a set of labels — to an **entire image**. The model receives the whole picture and outputs a probability for each possible class.

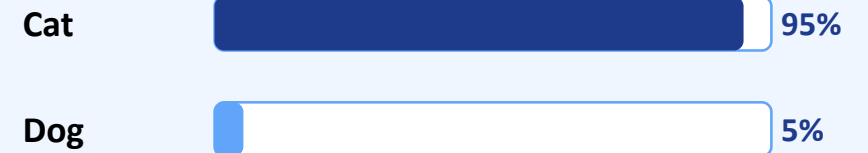
- **Single-label:** the image belongs to one class (e.g. Image → Cat)
- **Multi-label:** several classes can be predicted at once (e.g. Dog + Car + Person)
- **Limitation:** tells us WHAT is present, never WHERE it is

If the cat sat in the corner instead of centered, the prediction — and its confidence — would stay the same. Classification never looks at position.

INPUT IMAGE

IMAGE

↓ CNN



Predicted class: Cat (highest probability)



Object Localization

Main question: What is the object, and where is it?

Localization goes one step beyond classification: the model still predicts a class, but it also predicts the object's location, usually as a **bounding box**.

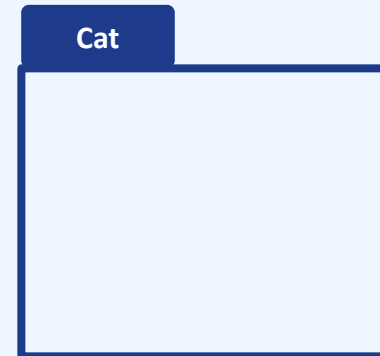
What is a bounding box?

A rectangle that surrounds the object, described by coordinates:

(x, y, w, h) — position (x, y) plus width and height

or $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$ — opposite corners of the box

The bounding box tells us approximately WHERE the object is located.



(x, y, w, h)

Classification vs. Localization

Same image — a different level of detail in the answer.



Classification

"What?"

- No location returned
- Output: class label only
- Usually assumes one main subject



Localization

"What + Where?"

- Gives a location
- Output: class + bounding box
- Usually focuses on one main object



Object Detection

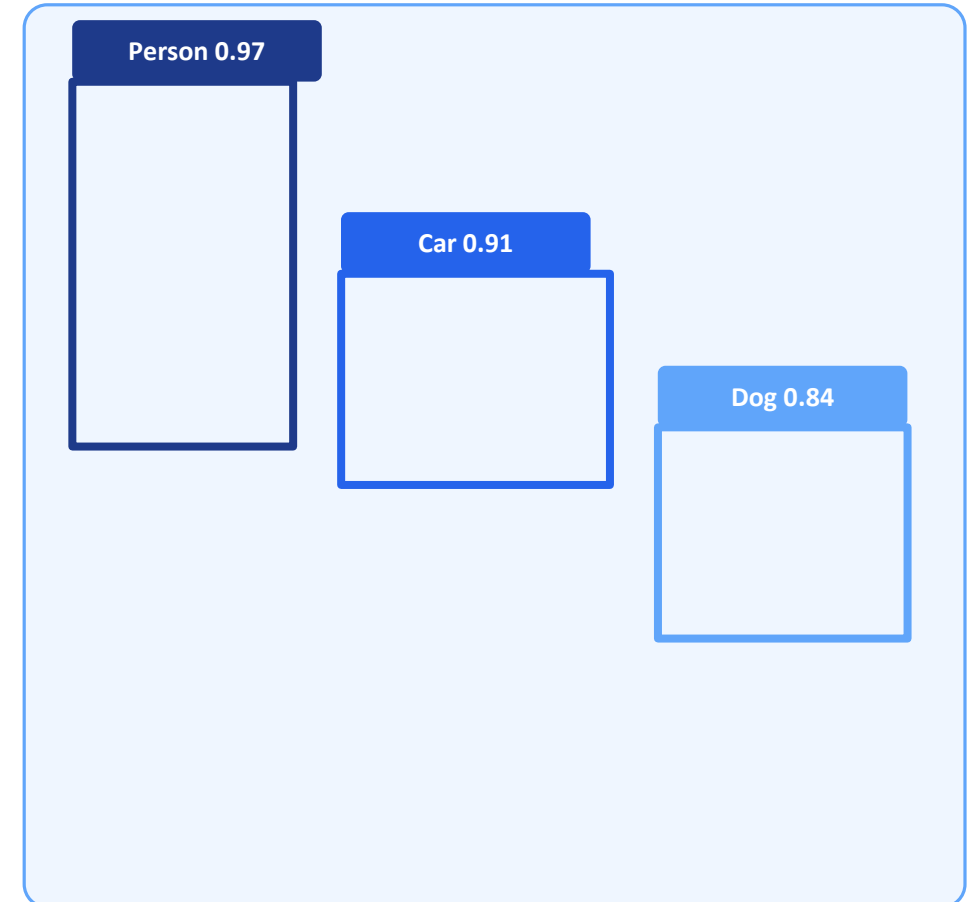
What objects are present, where are they, and how confident are we?

Detection extends localization to multiple objects in the same image. For each object found, the model returns three things:

1 Class what the object is (e.g. Person)

2 Bounding box where the object is (x, y, w, h)

3 Confidence how sure the model is (e.g. 0.97)



Examples of Object Detection Models

Detection can be thought of as localization performed for every object in the image.



YOLO

You Only Look Once

Fast, single-stage detection built for real-time use — autonomous driving, surveillance, robotics, traffic monitoring.



SSD

Single Shot MultiBox Detector

Another single-stage approach designed to perform detection using multiple feature-map scales to handle different object sizes.



Faster R-CNN

Region Proposal + Classification

A two-stage architecture: it proposes candidate regions first, then classifies and refines each one.



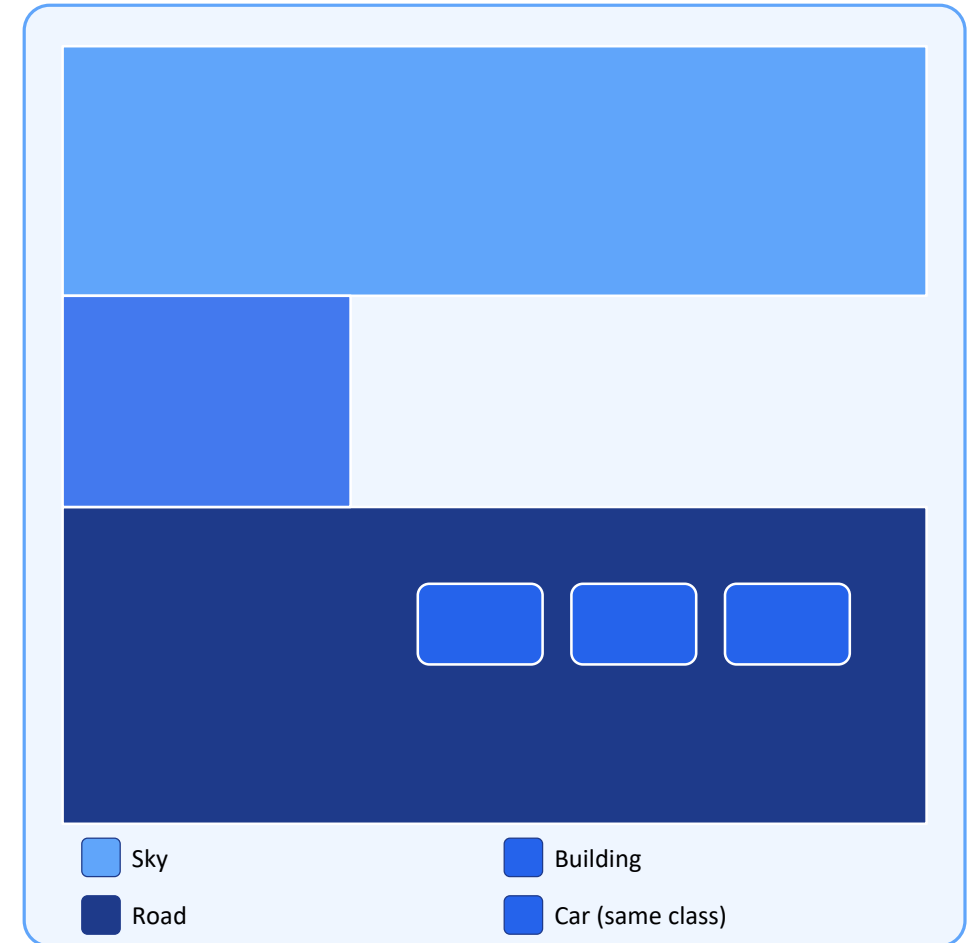
Semantic Segmentation

Main question: What class does each pixel belong to?

Instead of one label per image, semantic segmentation assigns a class to **every pixel**, producing a pixel-level class map rather than a rectangle.

*Detection gives a rectangle — much of it can be background.
Segmentation keeps only the pixels that truly belong to the object.*

Limitation: semantic segmentation knows the **class** of each pixel, but not which **individual object** it belongs to. Three separate cars are all labeled simply "Car" — not Car 1, Car 2, Car 3.





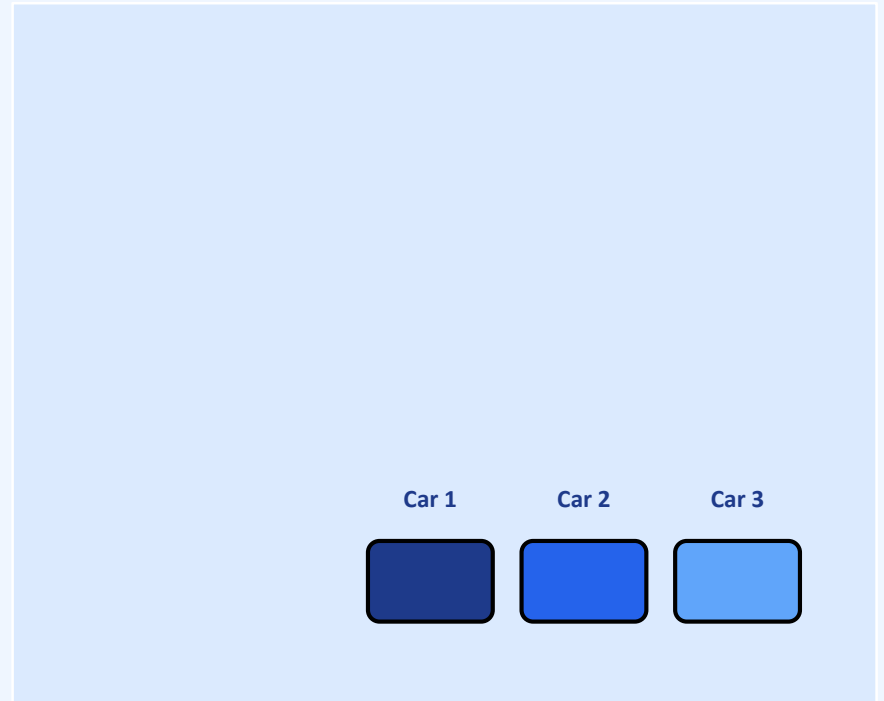
Instance Segmentation

Main question: Which individual object does each pixel belong to?

Instance segmentation combines object-level understanding with pixel-level segmentation: it finds **each individual object** and produces a separate mask for it — even when several objects share the same class.

A mask marks exactly which pixels belong to one specific object (1 = object, 0 = background), following its true shape.

Person 1	→	Mask 1
Person 2	→	Mask 2
Car 1	→	Mask 3
Car 2	→	Mask 4



Each car gets its own shade + mask — a separate instance.



What is a Mask?

Which pixels belong to an object and which pixels do not.



Putting It All Together

Task	Main Question	Output
Image Classification	What is in the image?	Class label(s)
Object Localization	What + where is one object?	Class + bounding box
Object Detection	What + where are multiple objects?	Class + box + confidence
Semantic Segmentation	What class is each pixel?	Pixel-level class map
Instance Segmentation	Which object owns each pixel?	Separate mask per object

The progression to remember:

What?

What + Where?

What + Where (many)?

Class per pixel

Object per pixel

DEEP LEARNING FUNDAMENTALS

Image Representation, Preprocessing & Augmentation

How a computer turns a picture into numbers a CNN can learn from

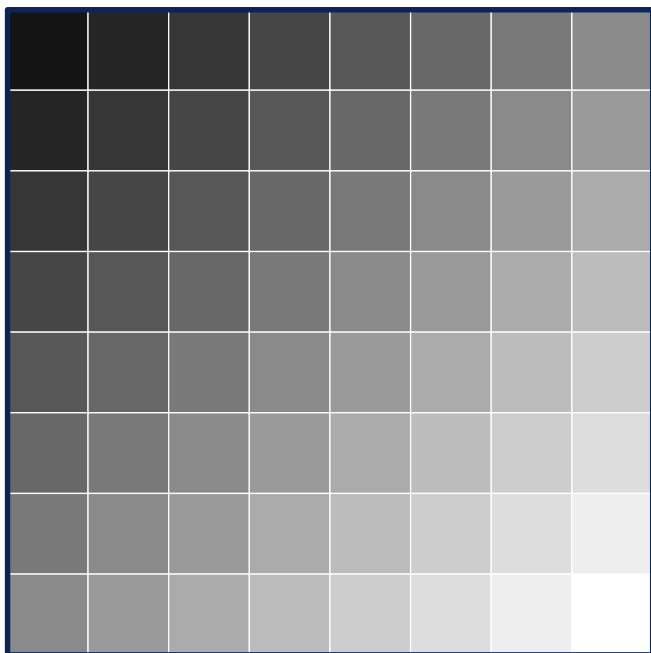
Representation

Preprocessing

Augmentation

What does an image look like to a computer?

A computer never sees “a cat.” It receives numbers arranged in a structured tensor.



$H \times W \times C$

H

Height number of pixel rows

W

Width number of pixel columns

C

Channels values stored per pixel

Example: $224 \times 224 \times 3 \rightarrow$ 224 pixels high, 224 pixels wide, 3 color channels

Grayscale vs. RGB — one number vs. three

GRAYSCALE

$H \times W \times 1$



0 = black

255 = white

One pixel → **128**

Each pixel stores a single intensity value — a shade of gray.

RGB

$H \times W \times 3$



R — Red

[255, 0, 0]



G — Green

[0, 255, 0]



B — Blue

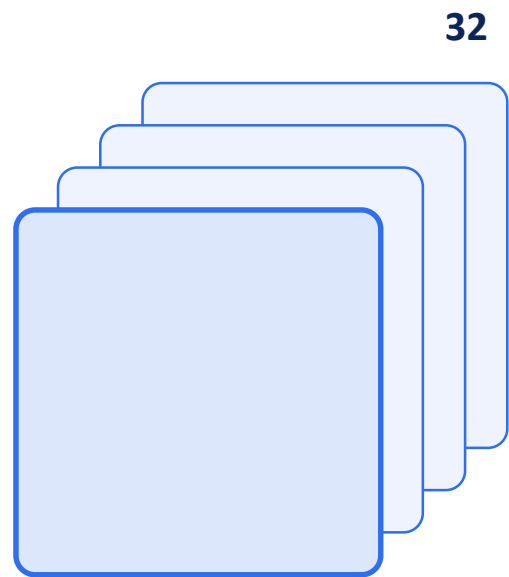
[0, 0, 255]

One pixel → **[128, 64, 200]**

Each pixel stores three values — how much red, green, and blue combine.

From one image to a batch: the 4-D tensor

Training rarely feeds one image at a time — models train on batches, adding a 4th dimension.



images in a batch

$$32 \times 3 \times 224 \times 224$$

32 → batch size

3 → channels

224 → height

224 → width

PyTorch

$N \times C \times H \times W$

TensorFlow / Keras

$N \times H \times W \times C$



This ordering trips people up constantly: swapping a PyTorch tensor into a TensorFlow model (or vice-versa) without permuting axes causes shape-mismatch errors — always check which convention your framework expects.

Real-world images are not ready for a CNN

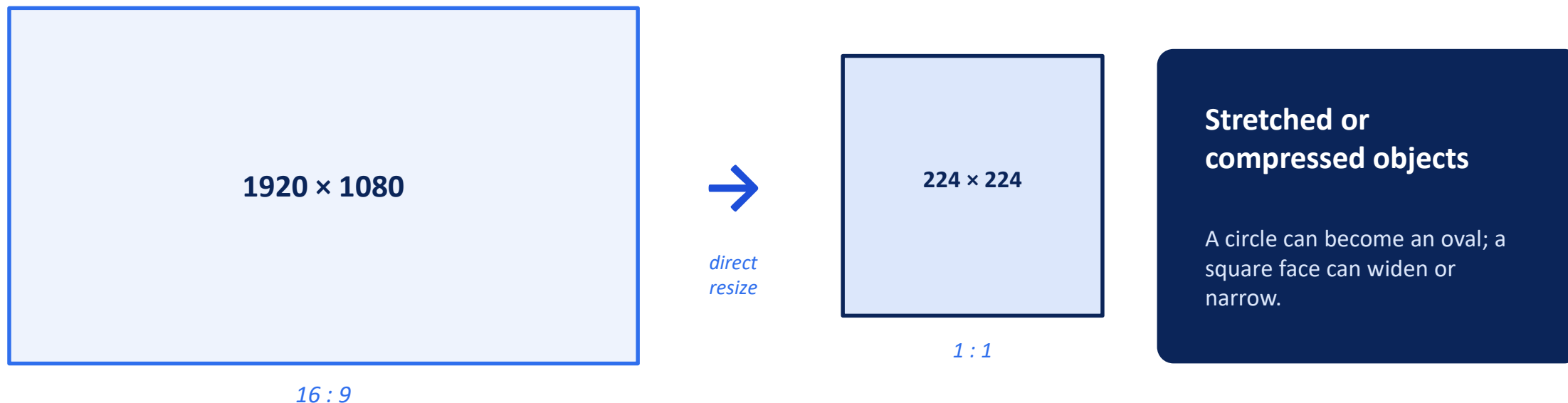
Raw images vary in ways a network can't handle directly. Preprocessing creates one consistent representation.



Raw Image → Resize / Crop → Color / Channel → Tensor → Scale / Normalize → CNN

The aspect-ratio problem

CNNs typically require a fixed input size — but forcing images into that shape can distort them.



Four ways to handle aspect ratio

1

Direct Resize

Stretch to target size.
Simple, but can distort the image.

2

Resize + Center Crop

Resize preserving ratio,
then crop the center region.

3

Random Crop

Pick a random region each
time — also acts as data
augmentation.

4

Padding

Preserve ratio and add
border pixels to reach the
target size.

Color and channel handling

Images arrive in different formats — and libraries don't always agree on channel order.

RGB

Grayscale

RGBA

Different channel orders

OpenCV loads images as

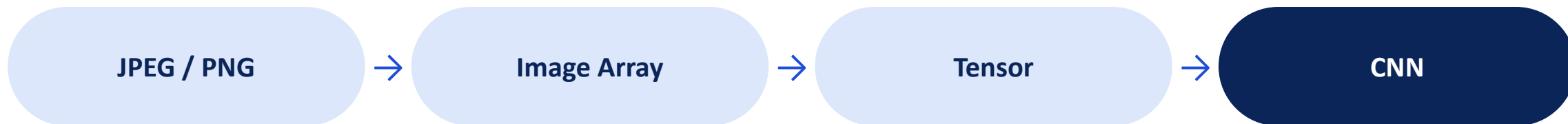
BGR

rather than

RGB

If you load with OpenCV but feed a pretrained CNN that expects RGB, convert channels first — a frequent, easy-to-miss bug.

From pixels to tensors, then rescale



Pixel Scaling

$$x' = x / 255$$

Raw pixel values in [0, 255] are rescaled to [0, 1].

Example: $128 / 255 \approx 0.502$

Why scale at all?

Putting inputs on a smaller, consistent numerical scale generally makes optimization easier — gradients behave more predictably and training converges faster.

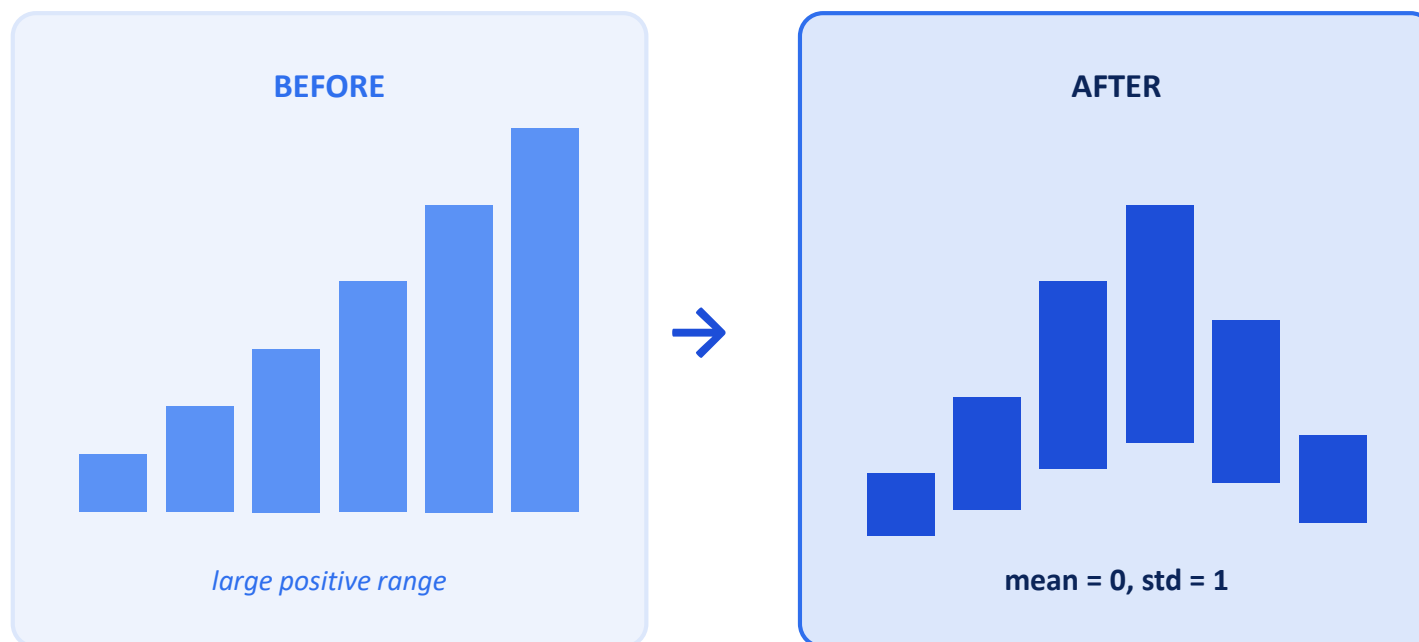
This is the step right before standardization.

Standardization: centering the distribution

After scaling to $[0, 1]$, we often standardize each channel using its mean and standard deviation.

$$\mathbf{x}' = (\mathbf{x} - \mu) / \sigma$$

μ = mean σ = standard deviation



Using a pretrained model? Normalize with the exact μ and σ it was trained with — otherwise inputs won't match what the model learned.

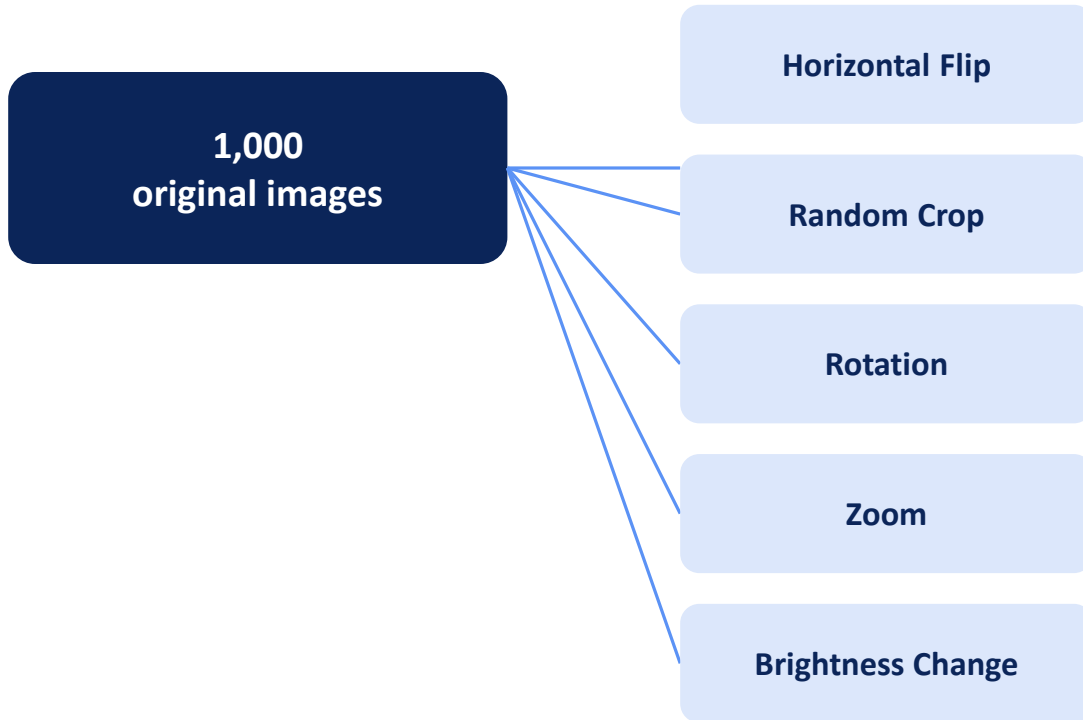
Preprocess every split the same way

The pipeline must be identical across train, validation, and test — only random augmentation is training-only.



Augmentation ≠ preprocessing

Preprocessing prepares the image. Augmentation deliberately creates variation in training data.



The goal isn't more files — it's teaching the model that the object is still the same even when its appearance changes realistically.

- 1 Reduce Overfitting** The model sees variation instead of memorizing examples.
- 2 Improve Generalization** Handles unseen images better.
- 3 Improve Robustness** Copes with rotations, crops, translations, and lighting shifts.

Common augmentation techniques

1

Horizontal Flip

Useful when left-right orientation doesn't change the class.

2

Random Crop

Forces recognition of the object from different spatial regions.

3

Rotation

Useful when objects naturally appear at different orientations.

4

Zoom

Helps the model handle different object scales.

5

Brightness / Contrast

Reduces dependence on one particular lighting condition.

The most important rule: preserve the label

An augmentation is only useful if the transformed image should still carry the same correct label.

✓ VALID

Cat

↓ *horizontal flip*

Cat

Label is unchanged — the flip is safe.

✗ RISKY

6

↓ *rotate 180°*

9 ?

For handwritten digits, this can change the meaning entirely.

Rule of thumb: choose augmentations according to the domain and task.

Detection & segmentation need label transforms too

Geometric transforms must be applied to the annotations too, or image and label stop matching.

Detection

Image + Bounding Box



Horizontal Flip



Flipped Image + Flipped Box

Segmentation

Image + Mask



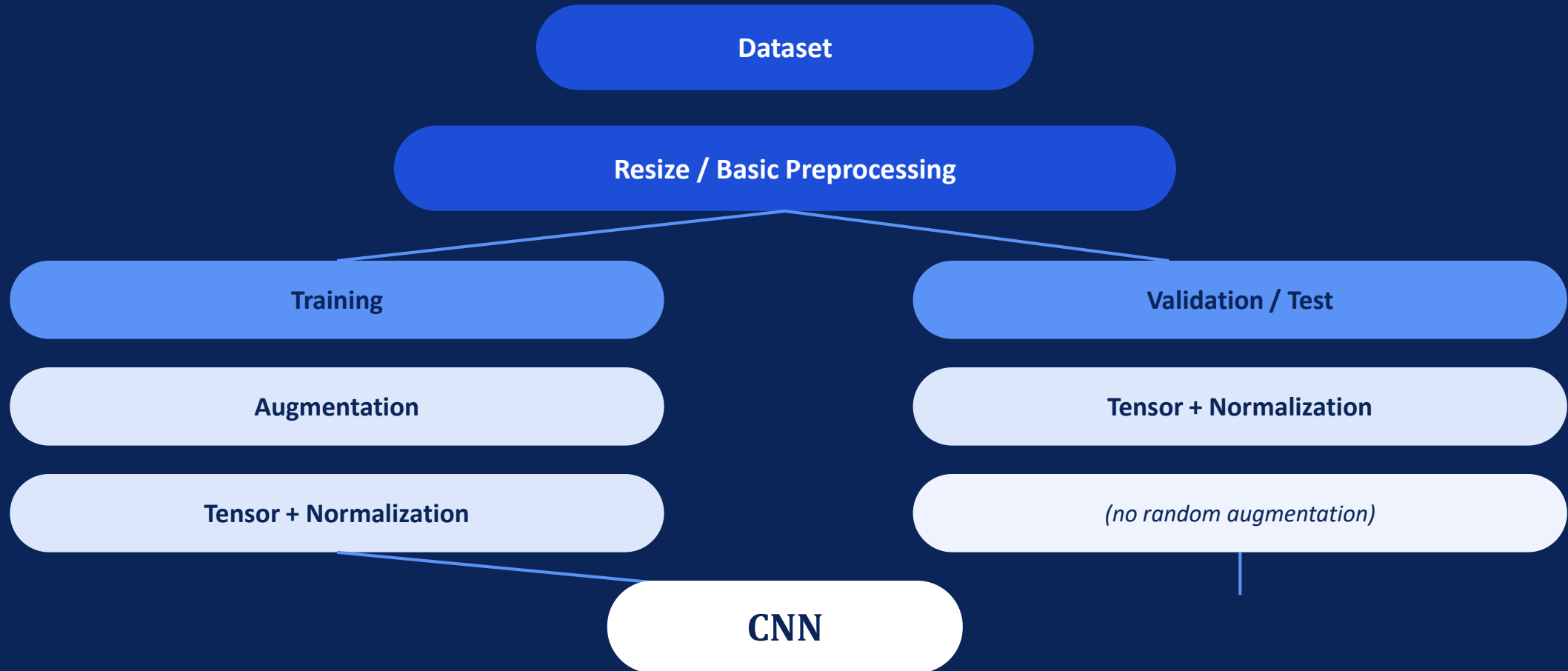
Rotation



Rotated Image + Rotated Mask

Classification only transforms the image. Detection and segmentation must transform the annotations in lockstep — otherwise image and label no longer correspond.

Raw image to trained CNN



DEEP LEARNING FUNDAMENTALS

Why Convolutional Neural Networks?

From fully connected layers to filters that share weights and exploit spatial structure

The Problem

Convolution

Feature Hierarchy

Feeding every pixel to a dense layer

Take one ordinary RGB image and connect it to a modest first layer — the numbers explode fast.

$$224 \times 224 \times 3$$

= 150,528 input values

×

$$1,000$$

neurons in layer 1

=

$$150,528,000$$

weights — one layer

150M+

parameters — in a single fully connected layer, before the network has learned anything useful.

Two problems this creates

1 Too Many Parameters

- Expensive computation
- More memory required
- Longer training time
- Greater risk of overfitting

2 Spatial Structure Ignored

Nearby pixels are related.

Neighboring pixels can form an edge, and multiple edges can form a shape.

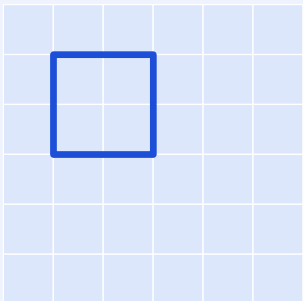
A fully connected layer treats every input independently — it never explicitly exploits this local structure.

A network designed for spatial data

Instead of connecting every pixel to every neuron, a CNN uses small filters that scan local regions.

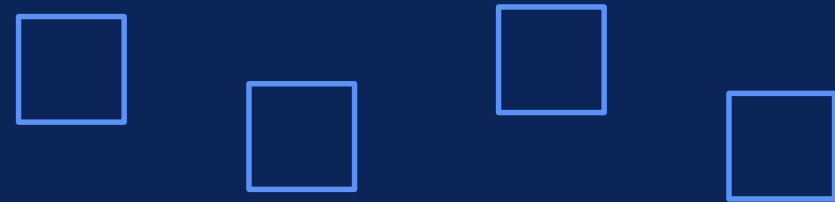
Local Connectivity

A filter looks at a small region of the image at a time, rather than the entire image at once.



Parameter Sharing

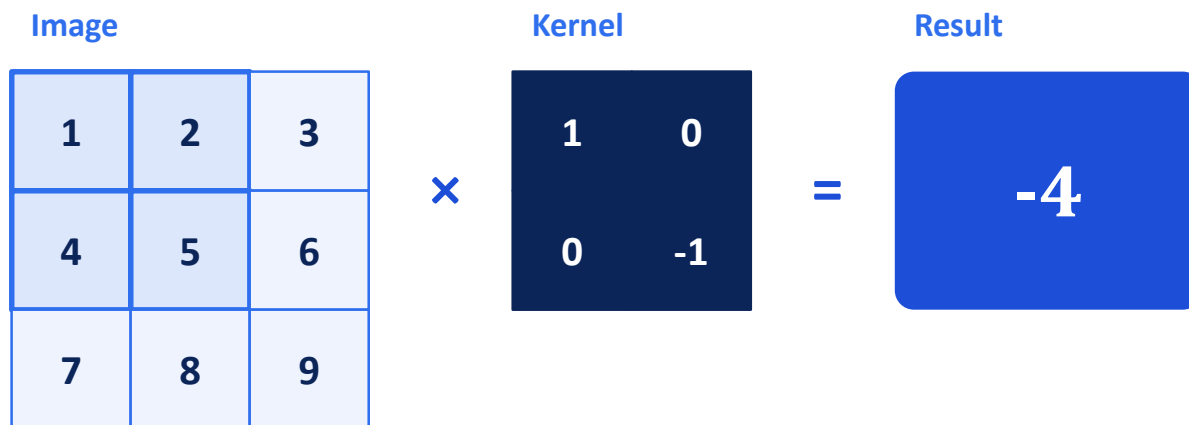
The same filter weights are reused at different locations across the image.



Together, these ideas dramatically reduce the parameter count while letting the network detect the same pattern wherever it appears.

Convolution: a filter sliding over pixels

A filter (kernel) is a small matrix of learnable weights that multiplies and sums a local patch of pixels.

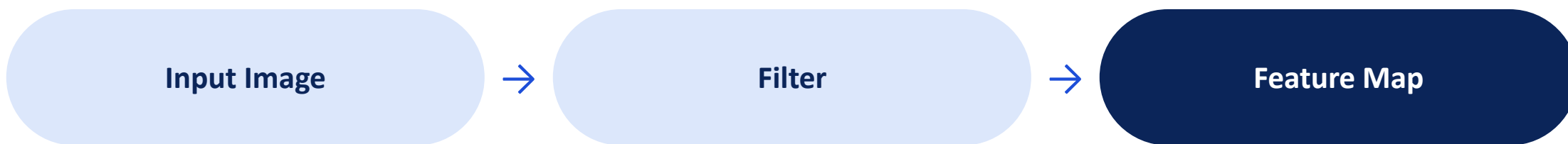


$$(1 \times 1) + (2 \times 0) + (4 \times 0) + (5 \times -1) = 1 - 5 = -4$$

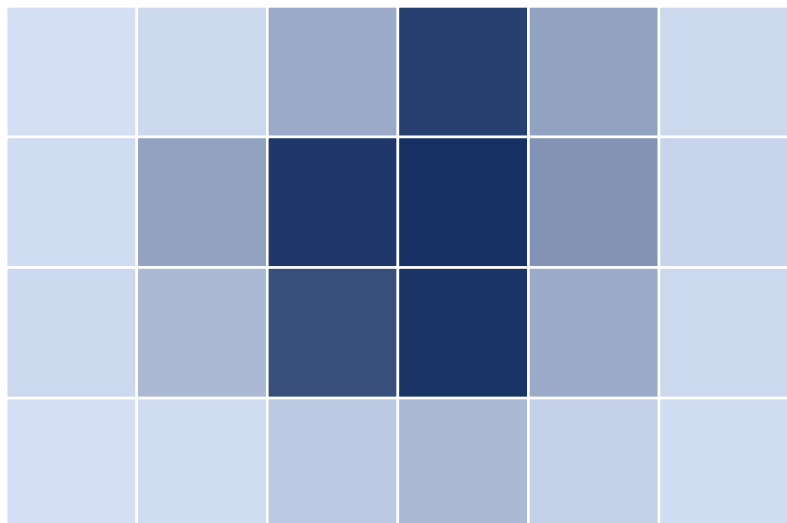
The kernel then slides to the next position and repeats this calculation across the whole image — the collection of outputs forms a feature map.

What is a feature map?

A feature map shows where the pattern a filter has learned appears across the image.



Darker cells = stronger activation where the pattern is detected



Darker activations → this filter fired strongly here, meaning its learned pattern (e.g. a vertical edge) was present in this region of the image.

One layer, many filters

A convolution layer doesn't learn just one filter — it learns many, each tuned to a different pattern.



32 filters → 32 feature maps, each detecting a different pattern — simultaneously, in one pass over the image.

Filters are trained, not hand-designed

Weights start as small random values, then training gradually shapes them into useful pattern detectors.



↻ repeated over many training iterations

Over many iterations, filters converge on weights that minimize the loss — this produces a hierarchy of learned features.

Early layers → edges

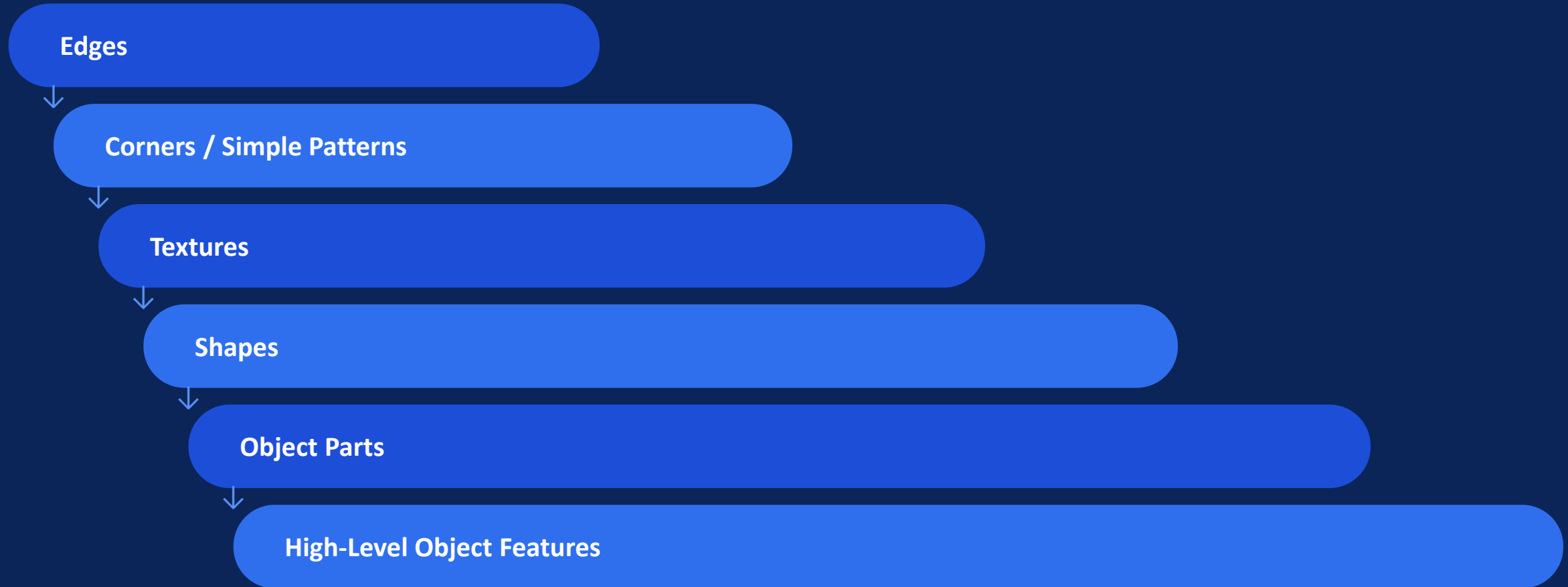
Middle layers → curves / textures

Deeper layers → eyes / nose / mouth patterns

Very deep layers → face-related structures

This hierarchy is an intuition, not a strict rule every CNN follows identically — exact features are learned from the training data.

A hierarchy of visual concepts



The complete CNN idea

Image

Small learnable filters scan local regions

Feature maps

Different filters detect different patterns

Deeper layers combine simpler patterns

Higher-level visual features

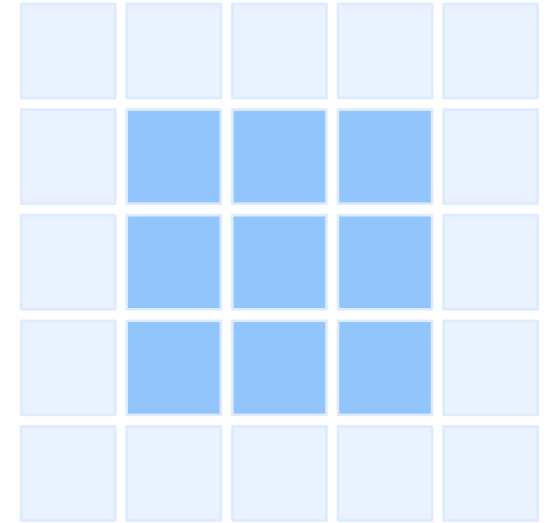
Prediction

CNNs don't learn a separate weight for every pixel-to-neuron connection — they learn reusable filters that detect useful patterns anywhere in the image.

CONVOLUTIONAL NEURAL NETWORKS

CNN Hyperparameters

Kernel size · Stride · Padding · Filters — and how they shape
the feature maps that flow through a CNN



Four Hyperparameters Control Convolution

They determine the size of output feature maps and what a layer can capture

1

Kernel Size

The spatial region examined at each step (e.g. 3×3, 5×5)

2

Stride

How far the kernel moves between positions

3

Padding

Extra border values added before convolving

4

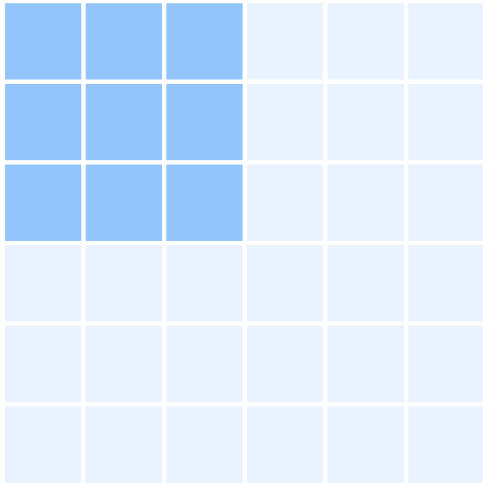
Number of Filters

How many distinct patterns the layer learns

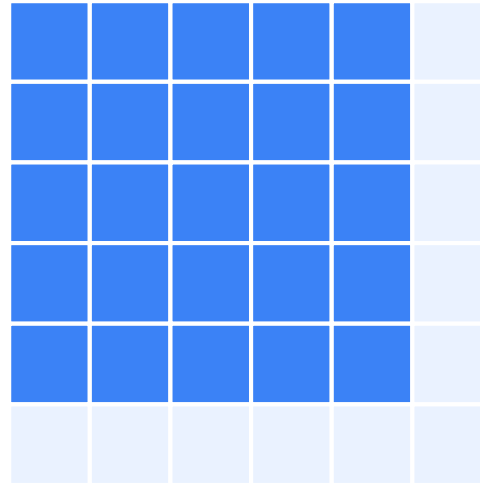
These directly affect the size of the output feature maps.

9.1 Kernel Size

The spatial region examined at each convolution step



3 × 3 kernel → 9 weights

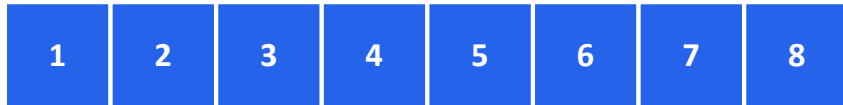


5 × 5 kernel → 25 weights

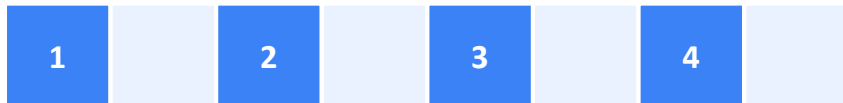
Why 3 × 3 is so common: it balances capturing local patterns, computational cost, and number of parameters. Modern CNNs stack multiple 3×3 convolutions instead of relying on large kernels.

9.2 Stride

How far the kernel moves between two consecutive positions



Stride = 1 (moves 1 pixel)



Stride = 2 (skips a pixel)

Key idea: kernel size controls how large a region is examined; stride controls how far we move between examinations. Larger stride → smaller output.

9.3 Padding

Extra border values added around the image before convolution

0	0	0	0	0
0	1	2	3	0
0	4	5	6	0
0	7	8	9	0
0	0	0	0	0

Zero padding around a 3×3 image

Valid Padding

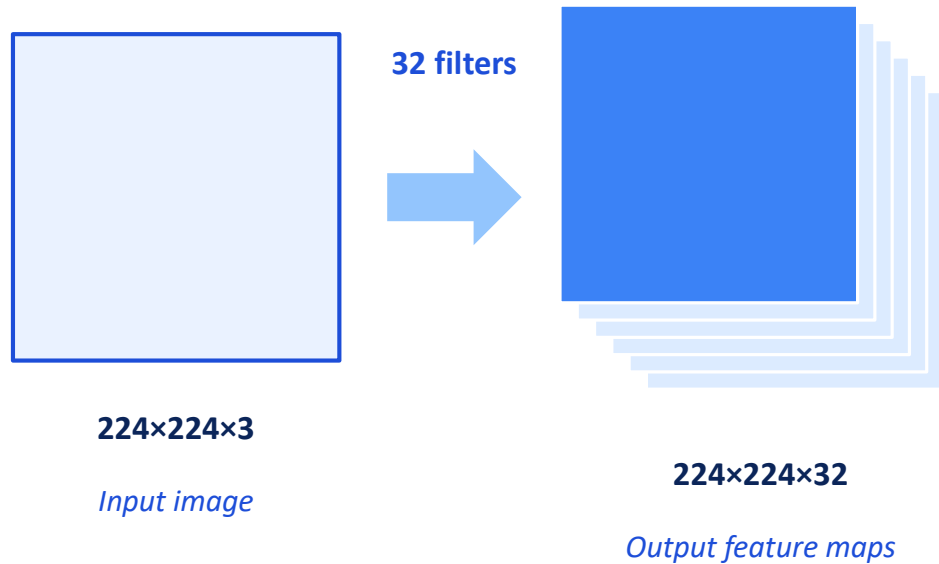
padding = 0 — no extra pixels added. Spatial dimensions shrink after every convolution.

Same Padding

Padding chosen so, with the right stride, output spatial size matches the input (e.g. $28 \times 28 \rightarrow 28 \times 28$).

9.4 Number of Filters

Each filter learns a different pattern and produces one output channel



Each filter produces one output channel.

32 filters → 32 feature-map channels

Every filter has its own learnable weights and can learn a different visual pattern — edges, textures, colors, and more.

Part 10 · Feature Map Size

The formula that determines spatial output size (applied to height and width)

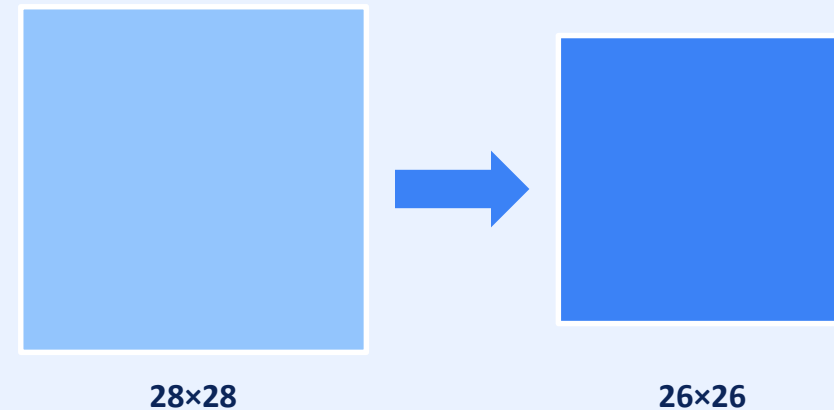
$$O = \lfloor (N + 2P - F) / S \rfloor + 1$$

- O** Output spatial size
- N** Input spatial size
- F** Kernel size
- P** Padding
- S** Stride

Worked Example

$N = 28, F = 3, S = 1, P = 0$

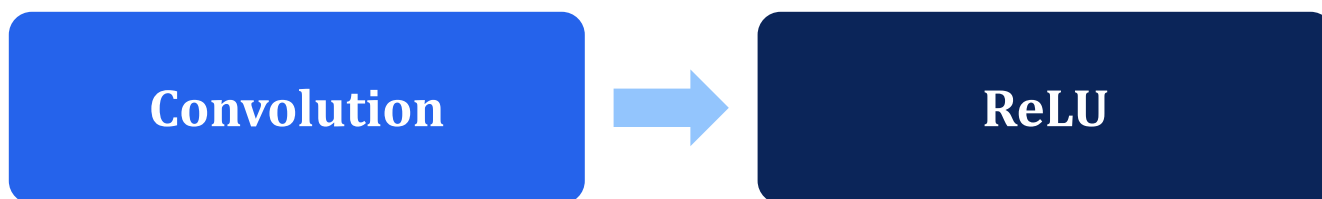
$$O = \lfloor (28 + 0 - 3) / 1 \rfloor + 1 = 26$$



With 32 filters, channels: 3 → 32, giving a final output of 26 × 26 × 32.

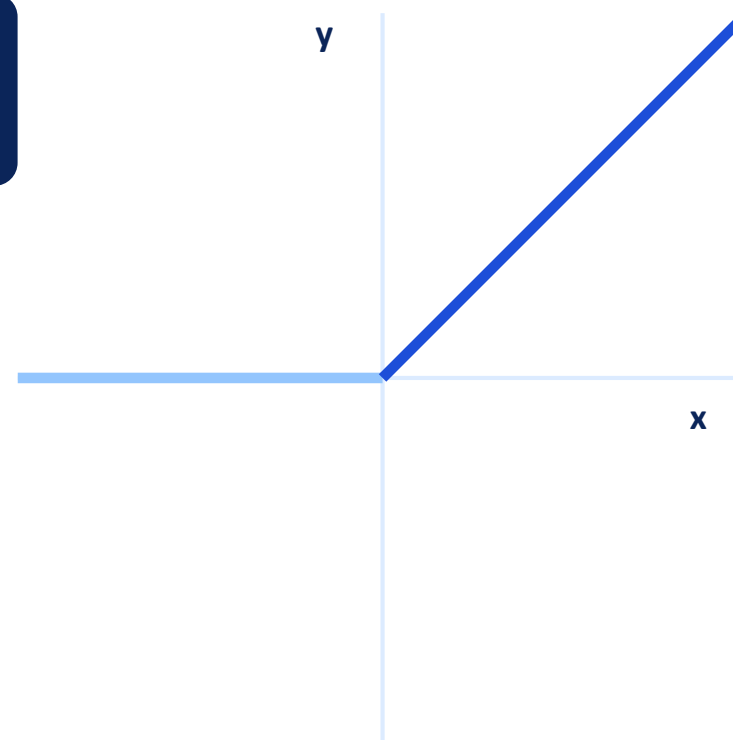
Part 11 · ReLU

Applied after convolution to introduce non-linearity



$$\text{ReLU}(x) = \max(0, x)$$

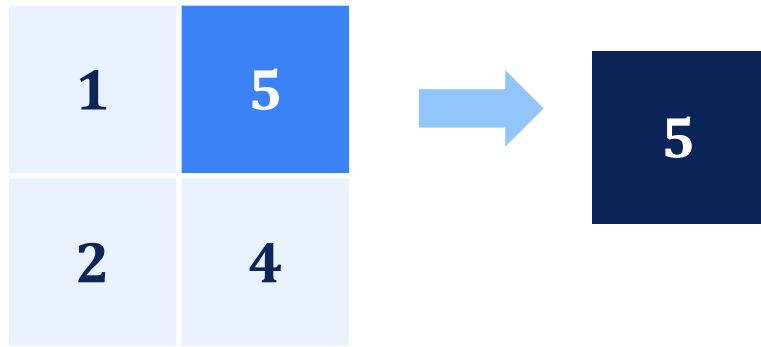
Negative convolution outputs become zero; positive values pass through unchanged. This lets successive CNN layers model far more complex relationships than a purely linear stack ever could.



ReLU activation curve

Part 12 · Pooling

A downsampling operation that reduces the spatial size of feature maps



$\max(1, 5, 2, 4) = 5$

Kernel = 2×2, Stride = 2 — non-overlapping windows

Why Pooling?

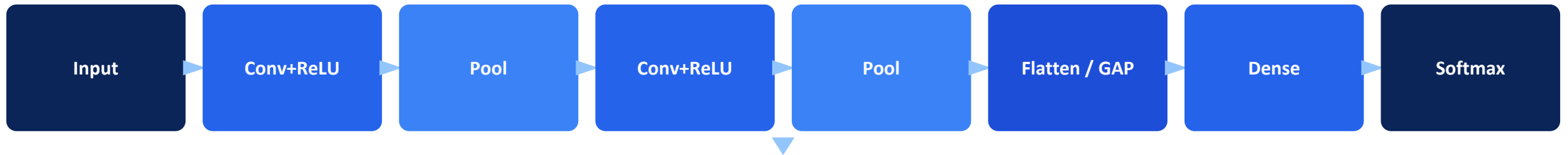
- Downsampling — reduces height & width (e.g. 28×28 → 14×14)
- Computational efficiency — fewer computations downstream
- Retains the strongest activation in each region
- Some tolerance to small spatial shifts in features

Max Pooling output = $\max(\text{region})$ — focuses on the strongest activation.

Average Pooling output = $\text{mean}(\text{region})$ — summarizes the whole region.

Part 13 · A Complete CNN Architecture

How the operations combine into a full pipeline



Prediction

What to notice as the representation moves through the network:

Spatial dimensions generally shrink (pooling / strided convolutions) while the number of feature channels increases — the network trades spatial resolution for a richer, more compact representation.

13.1 – 13.3 · Trading Resolution for Depth

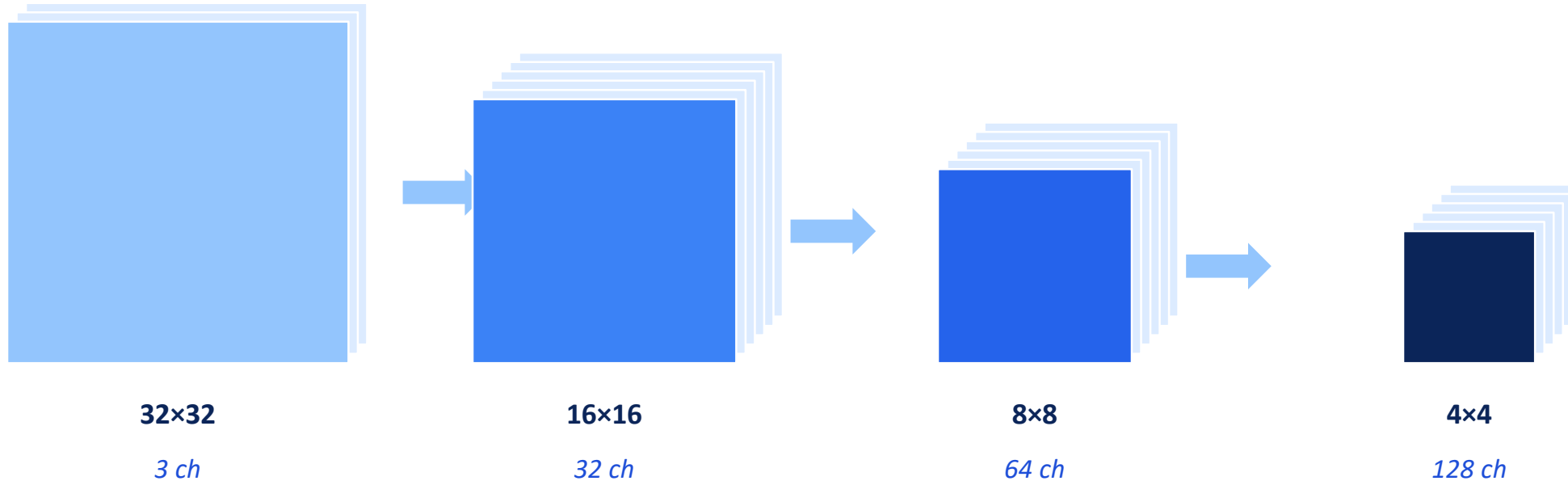
Spatial size shrinks while channel depth grows through the network

fewer pixels, more meaning per pixel

Spatial size ↓

Channels ↑

Deeper layers represent a larger variety of feature patterns — fewer pixels, more meaning per pixel.



Flatten vs. Global Average Pooling

Two ways to convert the final feature maps into a classifier-ready vector

Flatten

$4 \times 4 \times 128$



2048 values

Every spatial value from every channel is unrolled into one long vector, then fed to a dense layer. Preserves all spatial detail but creates many parameters in the classifier.

Global Average Pooling

$4 \times 4 \times 128$



128 values

Each feature map is averaged down to a single number — one value per channel. Dramatically reduces the parameters needed in the final classifier.

Key Takeaway

As the network goes deeper, spatial resolution generally decreases while the number of learned feature channels increases — producing a progressively more compact and expressive representation of the image.

- Kernel size → region examined per step
- Stride → step distance between positions
- Padding → controls border information & output size
- Filters → number of learned patterns / output channels
- Pooling → downsamples while keeping strong activations
- Flatten / GAP → bridges convolutional features to the classifier

CONVOLUTIONAL NEURAL NETWORKS

From Parameter Sharing to Transfer Learning

Receptive fields · classification · detection · segmentation · transfer learning

Parameter Sharing

One filter, reused everywhere.

When a filter scans an image, the network does not learn a new set of weights for every location — the same filter weights slide across the entire image.

- Filter at position 1, 2, 3 → SAME weights, not different ones
- A learned edge detector fires near the left, center, right, top, or bottom — no separate filter needed per location

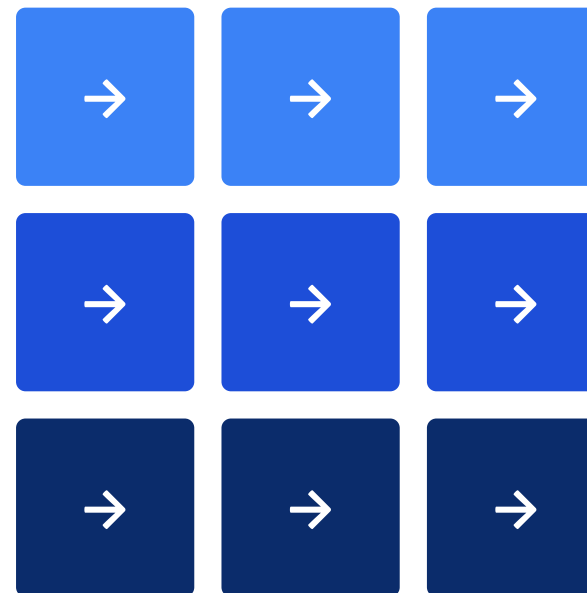
↓ Parameters

Far fewer weights to learn than one filter per pixel

Translation Robustness

A pattern is recognized wherever it appears in the image

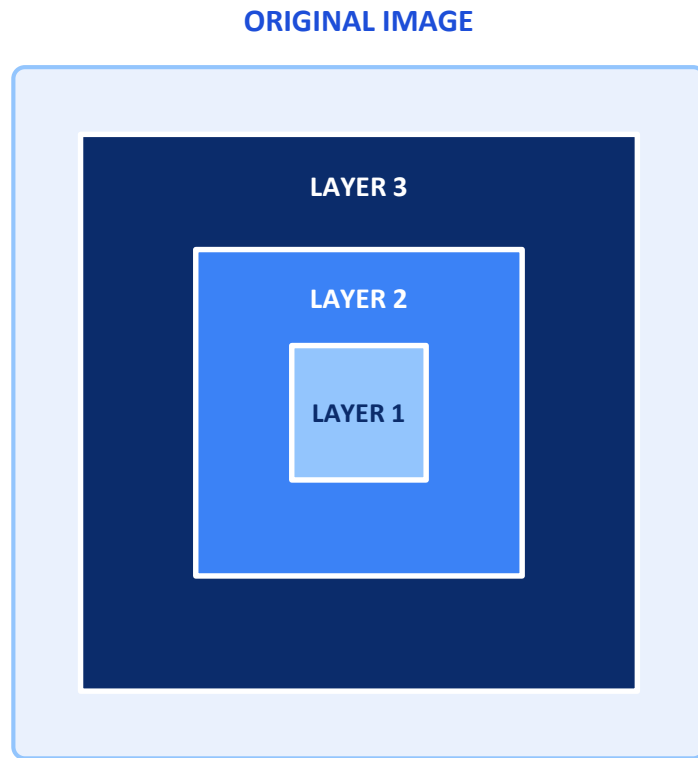
SAME FILTER ↓



The filter weights (e.g. an edge kernel) are identical in every tile above.

Receptive Field

The region of the original image that can influence one neuron's value — it grows as convolutional layers stack.



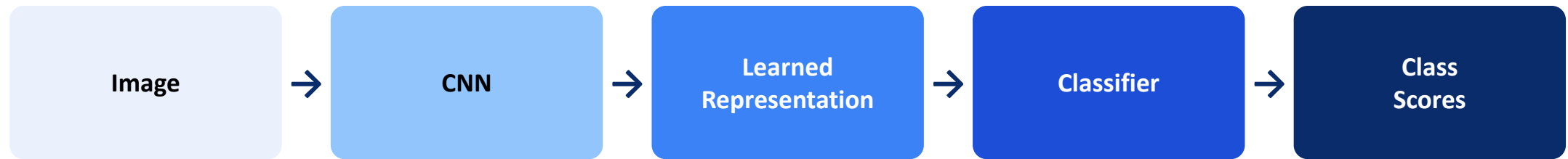
Small → larger → larger receptive field

What deeper layers can “see”

- 1 Local patterns
- 2 Edges / simple structures
- 3 Shapes
- 4 Object parts
- 5 Larger visual structures

Image Classification

The CNN acts as a feature extractor; a final classifier maps that representation to target classes.



Example prediction



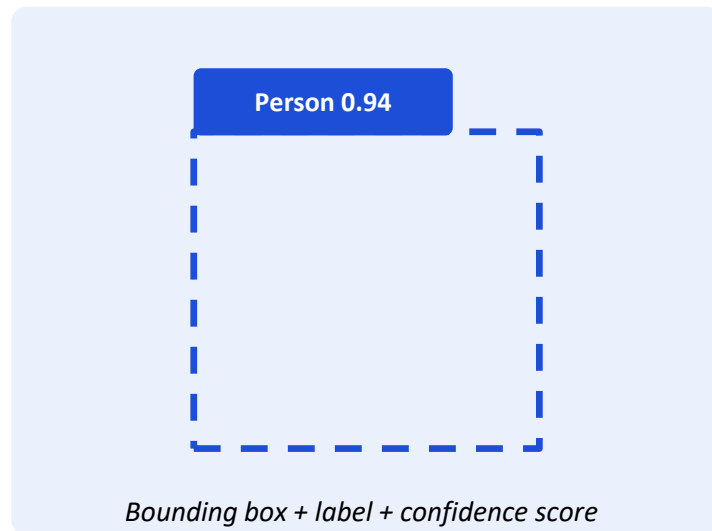
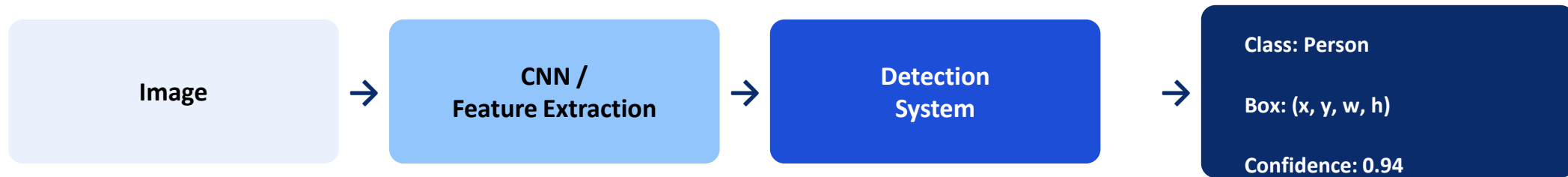
FINAL PREDICTION

Cat

Softmax converts the raw class scores into a probability distribution across all classes.

Object Detection

Instead of one label per image, detection predicts a class, box, and confidence for every object found.



YOLO

Efficient, often real-time detection

SSD

Single-stage; predicts objects at different spatial locations & scales

Faster R-CNN

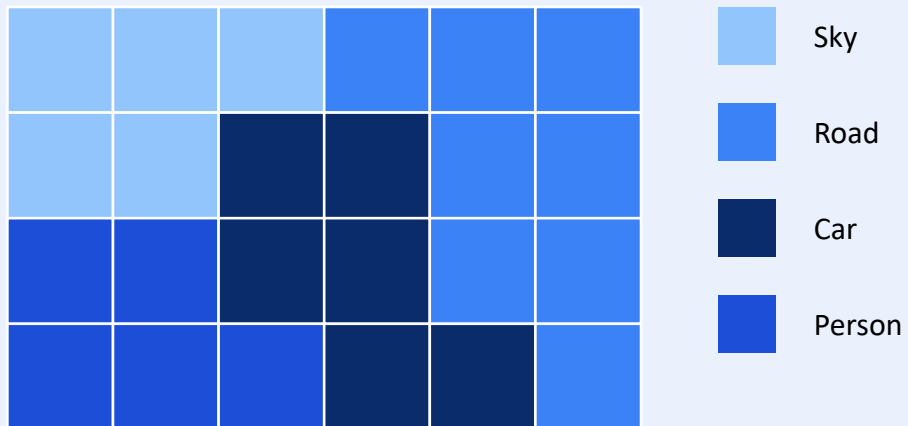
Two-stage: generates region proposals, then classifies & localizes

Image Segmentation

CNNs can predict a label for every pixel, not just one label for the whole image.

SEMANTIC SEGMENTATION

Every pixel gets a class label.



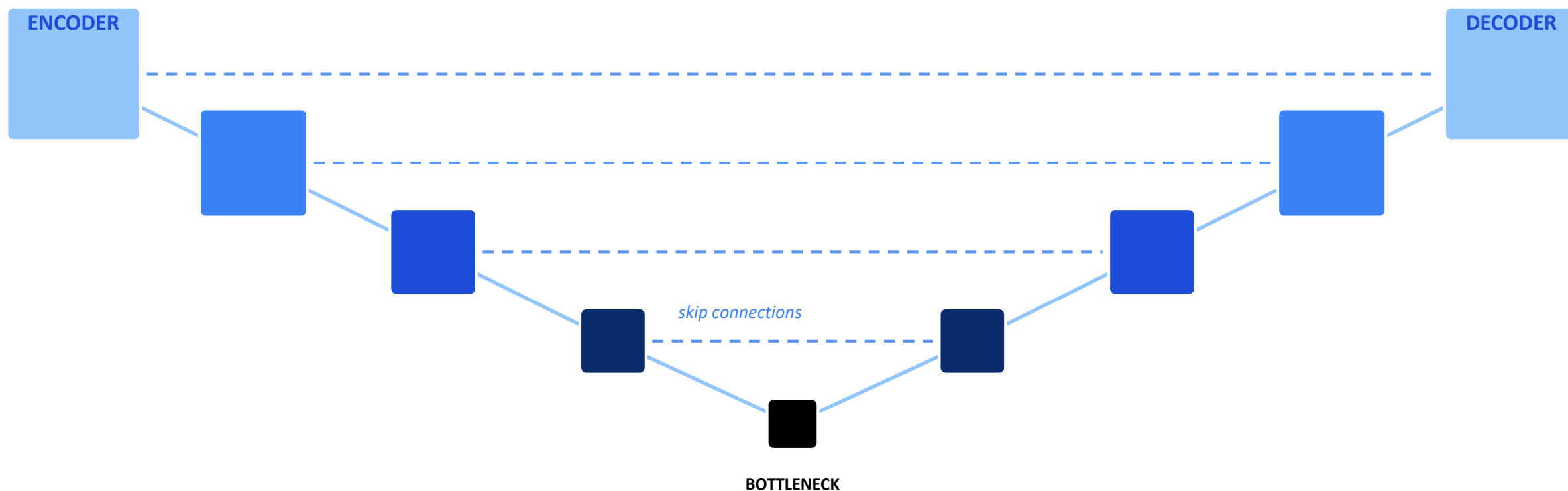
INSTANCE SEGMENTATION

Each individual object gets its own mask.



U-Net

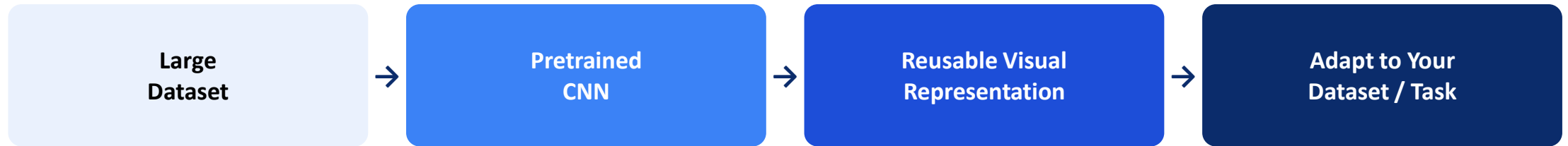
A U-shaped encoder–decoder network built for precise, pixel-level segmentation.



Skip connections feed earlier, high-resolution encoder features straight into the decoder — recovering precise object boundaries.

Transfer Learning

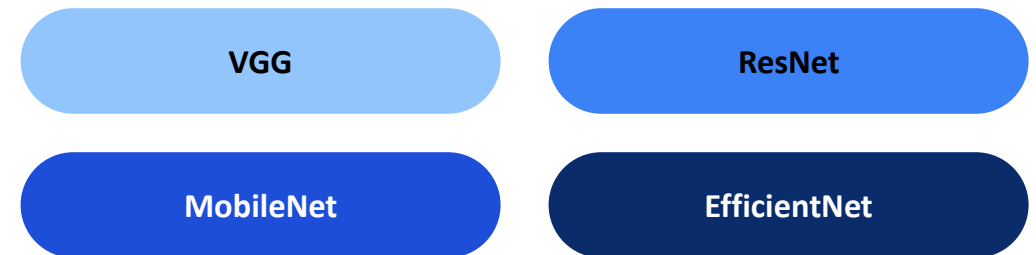
Reuse a CNN already trained on a large dataset instead of training completely from scratch.



Example: replacing the final layer



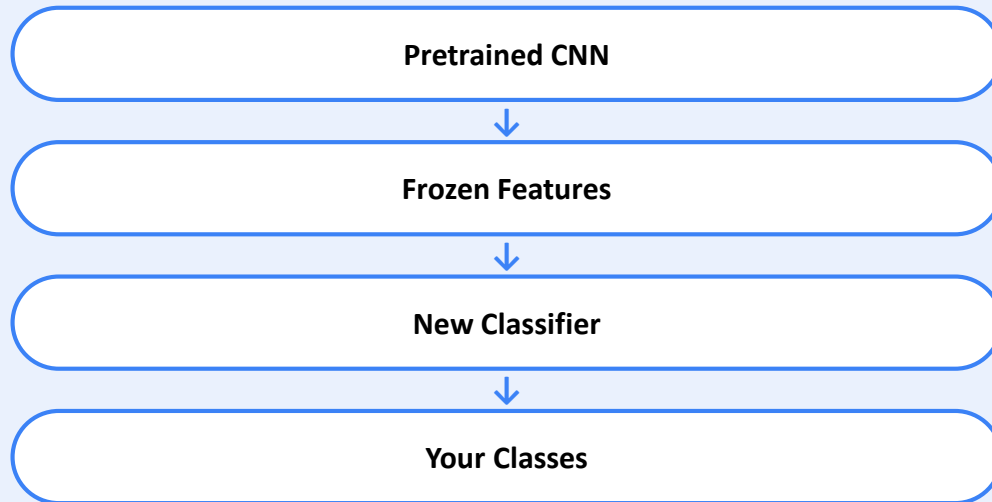
Common pretrained CNN families



Different image datasets share low- and mid-level patterns — edges, textures, curves, shapes — so a pretrained network is a useful starting point.

Two Transfer Learning Strategies

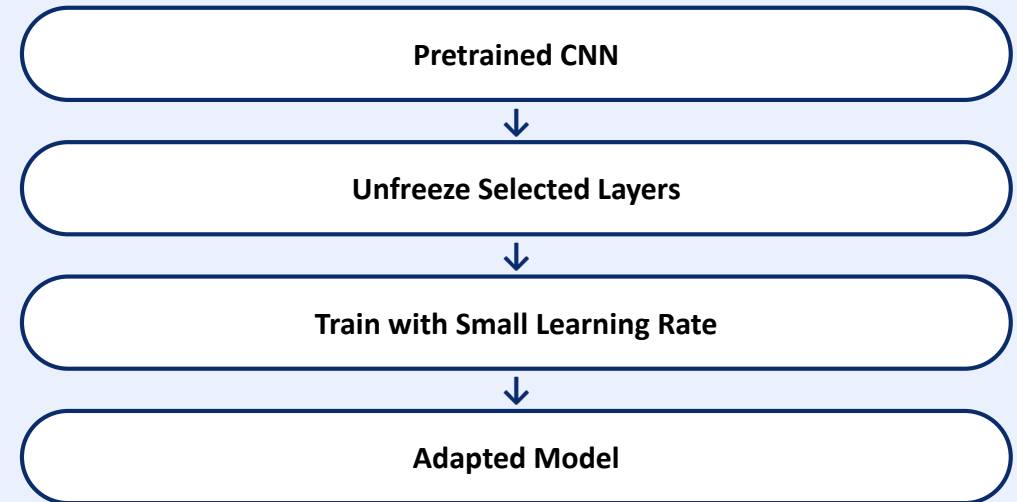
Feature Extraction



Keep most of the pretrained CNN fixed; train only a new classifier on top.

Best when your dataset is small or the existing features already fit well.

Fine-Tuning

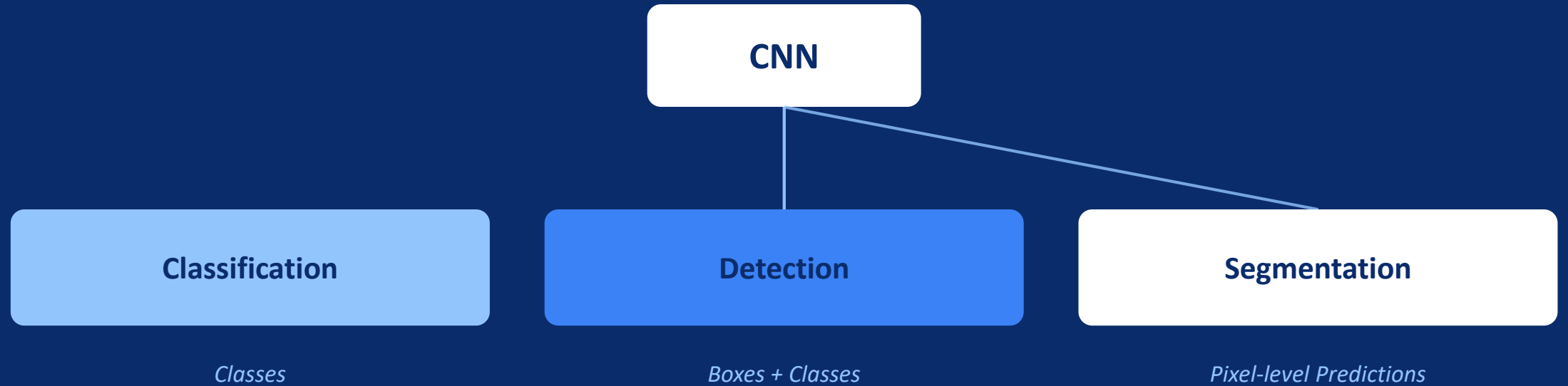


Allow some or all pretrained layers to keep updating on your new dataset.

Best when you want the representation to adapt further to your domain.

THE BIG PICTURE

What a CNN Can Do



+ Transfer Learning lets every one of these reuse a representation learned once, on a large dataset.